

Information Processing 1 — Week 13 exercises

Projects can be submitted the same week or the following week with no penalty.

New projects will be added each week. Each project is worth a certain number of points. You can score up to 50 points in total from all the projects you complete.

When you finish a project, **immediately** ask an instructor to grade it. If you are stuck or have any questions then **raise your hand** and an instructor will help you.

1 Guess the computer's number [5 points]

Task:

- the computer picks a random number between 1 and 100
- you guess the number, the computer tells you “too high” or “too low”
- when you guess correctly the computer prints the number of guesses you made
- you can then choose whether to play again

1.1 Example session

I have thought of a number between 1 and 100.

Try to guess the number.

Your guess: 50

Too high.

Your guess: 25

Too low.

...

Your guess: 26

You guessed the number in 7 attempts.

Play again (yes/no): y

I have thought of a number between 1 and 100.

Try to guess the number.

Your guess: █

1.2 Program structure

You could use two loops, one nested inside the other. The inner loop controls the game play, the body running for each guess typed in by the user. The outer loop repeats once per game, until the user chooses not to play again. The pseudo-code might look like this:

```
let playing be true
```

```
while playing:
```

- let *count* be 0
- let *number* be a random integer between 1 and 100
- repeat:

- let *guess* be the user's guess
- increase *count* by 1
- if *guess* < *number* then print "too low" and continue the loop from the start
- if *guess* > *number* then print "too high" and continue the loop from the start
- print "correct" and the value of *count*
- break out of the (inner) loop
- ask the user whether to play again
- if the response is "no" then break out of the (outer) loop

1.3 Random numbers

The `random` module provides the function `random.randint(low, high)` that returns a random integer in the range `low` to `high` (inclusive).

1.4 User input

The built-in `input(prompt)` function returns a string. For the user's guess you will have to convert this to an integer before you can compare it with the computer's random number.

After correctly guessing the number, you ask the user whether to play again. Their response might be "yes" or "Yes" or "y" or "no" or "n" or "No" or "NO". Your program is more 'user friendly' if it accepts any of those inputs and interprets them as yes/no appropriately.

To illustrate one way to do this, consider a program that needs to accept an answer of "true" or "false", or any prefix of those words, with any capitalisation. One way to organise the user input and test would be to write a function that accepts "true" or "false" as input and returns True or False accordingly, like this:

```
def getTrueFalse(prompt):
    while True:
        # valid response not yet received
        answer = input(prompt + " (true/false): ")
        answer = answer.lower()
        # ignore case differences
        if "true".startswith(answer):
            # any prefix of "true" is OK
            return True
        if "false".startswith(answer):
            # any prefix of "false" is OK
            return False
        print("I don't understand you; please type TRUE or FALSE.")

while not getTrueFalse("You understand how this works"):
    print("Read the code some more, and then answer again.")
```

Based on this example you might write a function called `getYesNo(prompt)` that does for "yes" and "no" what `getTrueFalse()` does for "true" and "false".

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

2 The computer guesses your number [5 points]

Task:

- you pick a random number between 1 and 100
- the computer guesses your number, you respond with “high” or “low” or “correct”
- when the computer guesses correctly it tells you the number of guesses it took
- you can then choose whether to play again
- if you cheat then the computer should detect that and complain

2.1 Example session

Think of a number between 1 and 100.

I will try to guess it.

Each time I guess, tell me if I am HIGH, LOW, or CORRECT.

My guess is: 50

Am I HIGH, LOW, or CORRECT: **h**

My guess is: 25

Am I HIGH, LOW, or CORRECT: **l**

My guess is: 37

Am I HIGH, LOW, or CORRECT: **h**

My guess is: 31

Am I HIGH, LOW, or CORRECT: **l**

My guess is: 34

Am I HIGH, LOW, or CORRECT: **c**

I got it in 5 attempts.

Play again (yes/no): **n**

2.2 Program structure

This can be similar to the previous project. An outer loop repeats once per game (until the user decides not to play again). An inner loop repeats once for each guess made by the computer along with the processing of the user's response.

2.3 Guessing numbers

The most efficient algorithm for guessing the user's secret number is a *binary search*. This algorithm keeps two variables that contain the **low** and **high** limits of the range in which the secret number is believed to be.

Each time around the game loop, the computer's **guess** should be a number half way between **low** and **high**. If the user says that **guess** is “high” then the computer knows that the secret number must lie in the range **low** to **guess-1** and updates **high** accordingly. Similarly, if the user says that **guess** is “low” then the computer knows that the secret number must lie in the range **guess+1** to **high** and updates **low** accordingly.

Here is an example of how the computer discovers the user's secret number 34 in five guesses.

		computer's guess	user's	computer's
lo	hi	$(lo+hi)//2$	response	update
1	100	50	high	hi = 49 (guess-1)
1	49	25	low	lo = 26 (guess+1)
26	49	37	high	hi = 36
26	36	31	low	lo = 32
32	36	34	correct	

(Note that **lo** and **hi** converge on the user's secret number. If ever **hi** becomes lower than **lo** it might indicate that the user is lying about their secret number.)

2.4 User input

You can handle “high”, “low”, and “correct” in the same way as “yes” and “no” are handled by the `getYesNo()` function in the previous project (see Section 1.4).

For the final question, whether to continue playing, you could reuse the `getYesNo()` function unmodified.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

3 Function plotting [8 points]

Task:

- let the user enter any mathematical expression in terms of x
- let the user set a minimum x value and a maximum x value, as expressions
- have the computer plot the user's function $y = f(x)$ between $x_{\min}$ and $x_{\max}$

3.1 Example session

Figure 1 shows a session in which the user plots the function $\sin(x)$ from $x = -2\pi$ to $x = 2\pi$ and then changes the function to $\cos(x)$, x and the plotting range to $x = -\pi$ to $x = \pi$, before plotting the new function.

3.2 Program structure

One user-friendly way to structure this kind of program is as a menu-driven activity. The program continually presents a menu of possible actions (including current values of variables, where appropriate) and asks what to do, expecting a number or a keyword as a response from the user. Based on the user's

```

1) change function: sin(x)      1) change function: cos(x)
2) change minimum: -2*pi        2) change minimum: -pi
3) change maximum: 2*pi         3) change maximum: 2*pi
4) plot function                4) plot function
5) exit                        5) exit

choice: 4                       choice: 3
                                new minimum: pi

1) change function: sin(x)      1) change function: cos(x)
2) change minimum: -2*pi        2) change minimum: -pi
3) change maximum: 2*pi         3) change maximum: pi
4) plot function                4) plot function
5) exit                        5) exit

choice: 1                       choice: 4
new function: cos(x)

1) change function: cos(x)      1) change function: cos(x)
2) change minimum: -2*pi        2) change minimum: -pi
3) change maximum: 2*pi         3) change maximum: pi
4) plot function                4) plot function
5) exit                        5) exit

choice: 2                       choice: 5
new minimum: -pi

```

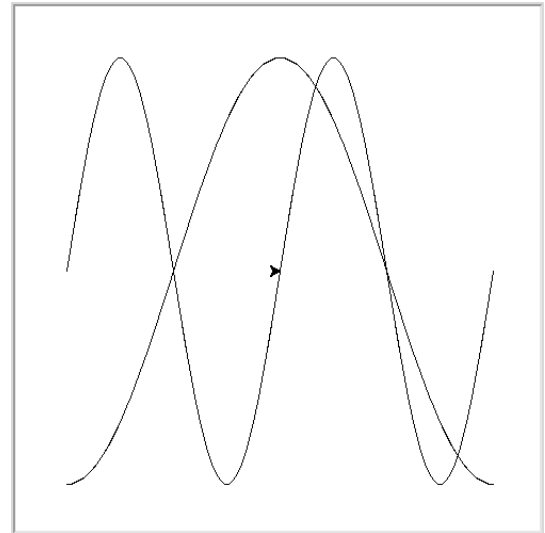


Figure 1: Example session for the 'function plotting' project.

response some action is performed, and then the menu is shown again. The program terminates when the action chosen by the user is 'exit'.

In this project the menu could display a set of actions that allow the user to

- change the function to be plotted (while showing the current value),
- modify the lower or upper limit on the x range to be plotted (showing current values),
- actually plot the function, or
- exit the program.

3.3 Converting expressions in strings into values

The x limits and the function to be plotted are entered by the user. Reading them with `input(prompt)` will give you strings, which are convenient for the menu's operation.

When it is time to plot the function you will have to convert the x values into numbers. The function `eval(s)` will evaluate the expression in the string `s` and return its value.

Similarly, to calculate the y value of the function for each x value in the range, you can use `eval(s)` inside a loop that sets `x` to a number of values between the low and high limits of the desired x range. The following short example program demonstrates this.

```

function = "x**2" # x squared
for x in range(-5, 6):
    y = eval(function)
    print("x =", x, "y =", y)

```

To allow the use of mathematical constants (such as `pi`) in your expressions, it would be user-friendly to import all of the `math` module into your program at the beginning.

```

from math import *
from turtle import *

def function(x): return sin(x) # the function to plot

xmin = -2*pi    # input range for function
xmax = 2*pi

setup(500, 500) # create a 500 x 500 window for plotting

ymin = function(xmin) # output value at first point
ymax = ymin

# calculate the actual range of output values ymin to ymax

xscale = (xmax - xmin) / 400 # scale input range to 400 pixels wide

for wx in range(401):        # window coordinates
    x = xmin + wx * xscale    # function coordinates
    y = function(x)           # function coordinates
    ymin = min(ymin, y)       # minimum y seen so far
    ymax = max(ymax, y)       # maximum y seen so far

if ymin == ymax:              # y is constant
    ymin = ymin - 1           # avoid divide by zero below

yscale = 400 / (ymax - ymin) # scale output range to 400 pixels high

up()                           # do not draw a line to the first point
for wx in range(401):         # window coordinates
    x = xmin + wx * xscale    # function coordinates
    y = function(x)           # function coordinates
    wy = (y - ymin) * yscale  # window coordinates
    goto(-200 + wx, -200 + wy) # draw line to next function point
    down()                    # draw lines starting from first point

up()
goto(0, 0) # park the turtle back at the origin

mainloop()

```

Figure 2: Plotting the function $\sin(x)$ from -2π to 2π in a 400-pixel square area with automatic scaling of width and height according to the specified input range and detected output range of the function.

3.4 Plotting a function

The `turtle` module is very convenient for plotting functions. Figure 2 shows an example of using a turtle to plot the function $\sin(x)$ between -2π and 2π .

The slight complexity of this program comes from having to scale the 400-pixel horizontal space in the window to 400 function input values evenly spaced between $x_{\min}$ and $x_{\max}$, and scaling the detected output range of the function to a 400-pixel vertical space in the window.

3.5 Hints

Storing the user's function as well as their chosen $x_{\min}$ and $x_{\max}$ as strings until the 'plot' action is chosen makes managing the menu easy.

Use `eval(s)` to convert $x_{\min}$ and $x_{\max}$ to floats when it is time to plot the function.

Use `eval(s)` to evaluate the user's function for each x between $x_{\min}$ and $x_{\max}$.

Use a `'try: ... except Exception as e: ...'` block to protect the program from nasty surprises when using `eval(s)` to evaluate either the x range limits or the function's value.

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

4 Finding the largest files [7 points]

Task:

- find (recursively) all files below a certain directory
- for each file, find the size of the file
- sort the results into order of increasing file size
- print each file name preceded by its size

(This kind of program is very useful for finding forgotten files that are wasting a lot of disk space!)

4.1 Example session

```
piumarta$ python3 largest-files.py /Volumes/Local/Sync/src/ccmeta
516      /Volumes/Local/Sync/src/ccmeta/Makefile
2773     /Volumes/Local/Sync/src/ccmeta/doc/main.aux
3842     /Volumes/Local/Sync/src/ccmeta/buffer.h
9770     /Volumes/Local/Sync/src/ccmeta/doc/main.log
16994    /Volumes/Local/Sync/src/ccmeta/object.c
23010    /Volumes/Local/Sync/src/ccmeta/doc/main.tex
85473    /Volumes/Local/Sync/src/ccmeta/doc/main.pdf
89908    /Volumes/Local/Sync/src/ccmeta/ccmeta.leg
128704   /Volumes/Local/Sync/src/ccmeta/ccmeta
```

4.2 Program structure

The program has three distinct sub-tasks to perform. To make it easier to test, each can be performed as a separate activity:

1. make a list containing the paths of all files below a certain point in the hierarchy
2. convert the list of paths to a list of sizes and filenames
3. sort the list and print each element

4.3 Hints

The first sub-task is to find all paths below a certain directory. A modified version of the recursive function we wrote in class to print all paths under a directory can be the basis for this. Let's call this function `getfiles()`. The functions

```
os.listdir(path),
os.path.isdir(path),
os.path.isfile(path), and
os.path.join(path, name)
```

will all be essential to the implementation of `getfiles()`. The `getfiles` function will have to be modified slightly to create a list of paths instead of printing them. For example, you could pass it a directory path as first argument and a list of files as the second argument. The `getfiles(dirpath, pathlist)` would therefore (recursively) extend the `pathlist` with all the filename paths it can find starting at the given `dirpath`. You will find it very convenient if `getfiles()` returns the list of file paths as its result.

The second sub-task is to prepend each file path in a list with the size of the file. The size of the file at `path` is returned by the function `os.path.getsize(path)`. A complication here is that you cannot just prepend the size to the path as a string. Consider, for example, three files "a", "b", and "c" whose sizes are 10, 11, and 100 bytes. If you prepend the size as a string and sort the list the order will be

```
10  a
100 c
11  b
```

because of the way strings are sorted into dictionary, not numerical, order. One solution to this is to convert the list of file paths into a list of tuples, where each tuple contains (`size`, `path`) and where `size` is an integer not a string. Sorting that list of tuples works as desired because the tuples will be sorted according to their first element, which is still an integer and not a string.

You might therefore want to make a function `getsizes(paths)` that converts a list of paths into a list of tuples containing a size and a path. The resulting list can be passed to `sorted(seq)` which will return a sorted list. A loop

```
for size, name in sorted(getsizes(getfiles(dirname, []))):
    print(size, path)
```

should then do almost exactly what you want.

The third task (printing the size and path neatly) can be accomplished by changing the separator in the above `print()` statement from a space to a tab character (`'\t'`).

4.4 Command-line arguments

If you want to make your program more useful by reading the `dirname` from the command line (as in the example session above) then here is how to do it.

The `sys` module provides a variable called `sys.argv` containing all the words that were typed on the command line, including the name of the program but excluding the name of the Python interpreter if it was specified. To obtain just the arguments (without the name of the program) you can use a slice to remove the first element of `sys.argv` like this:


```
import sys
print("I received these arguments:", sys.argv[1:] )
```

▷▷ Ask an instructor to check and record your work now (before you run out of time).